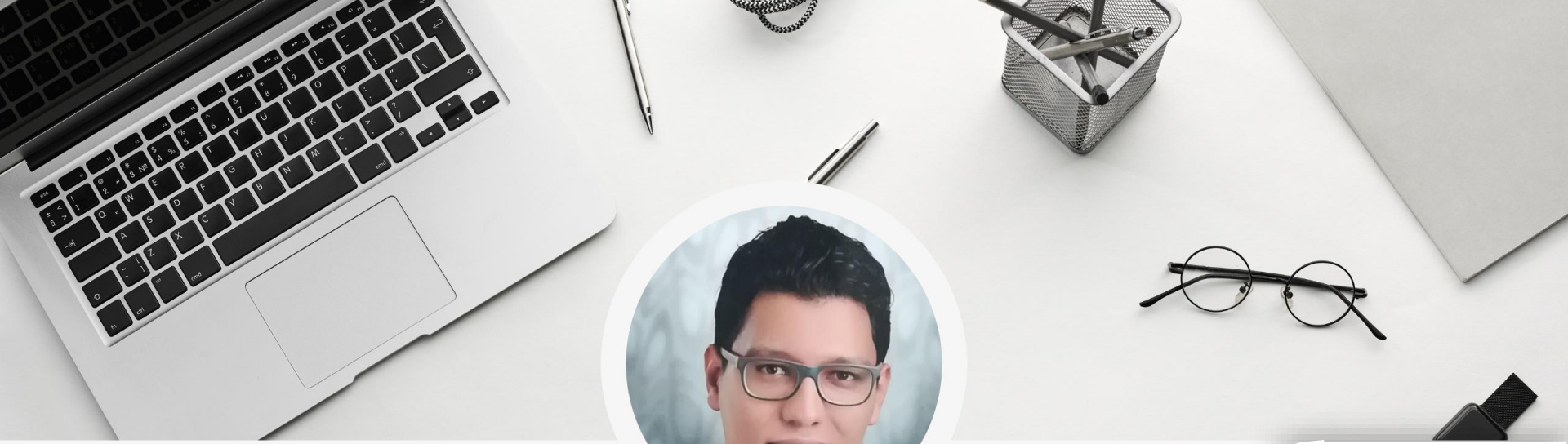
SOLID Principles



- **Software Engineering:** Professional experience **since 2010**.
- **Training & Development:** Active instructor and mentor **since 2012**.
- **ITI Technical Trainer:** Certified technical trainer **since 2014**.
- **Master's Degree in Agile Project Management, Maryland University**.
- **MBA in Project Management**, Arab Academy.
- **TOT (Training of Trainers)**.

Course content

01
SOLID Prerequisites

02
Software Problems, Solutions

03
SOLID Definition

04
Why Use SOLID

05
Coupling Concept

06
OOP,SOLID

07
SOLID Principles

08
OOP-SOLID-Design Pattern

01-SOLID Prerequisites

OOP

02

Software Problems and Solutions

Software Problems and Solutions :

Software Problems :

- Putting more **stress** on classes by assigning more responsibilities to them. (A lot of functionality not related to a class.)
- Forcing the classes to **depend on each other**. If classes depend on each other (in other words, tightly coupled), then a change in one will affect the other.
- Spreading **duplicate code** in the system/application.

Solutions :

- Choosing the **correct architecture** (MVC, 3-tier, Layered, MVP, MVVP, and so on).
- Following **Design Principles**.
- Choosing the correct **Design Patterns** to build the software based on its specifications.

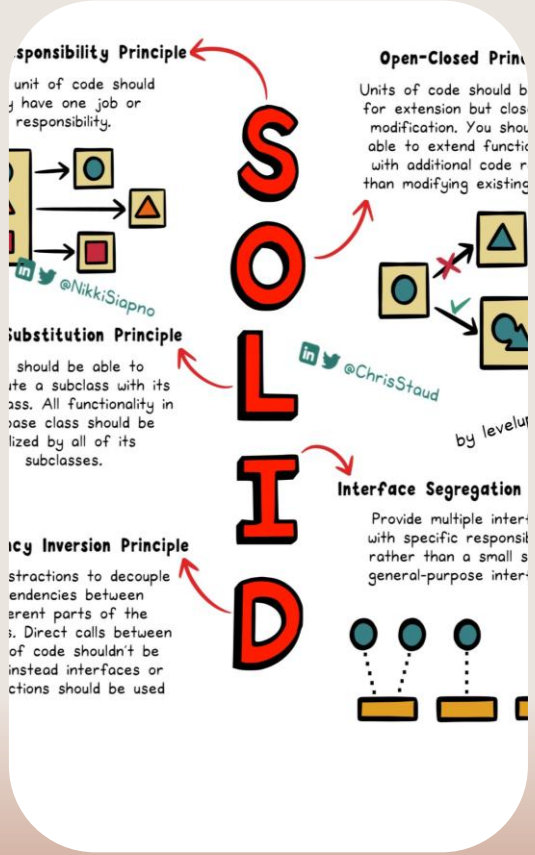
We go through the Design Principles first and will cover the rest soon.

03-SOLID Defination

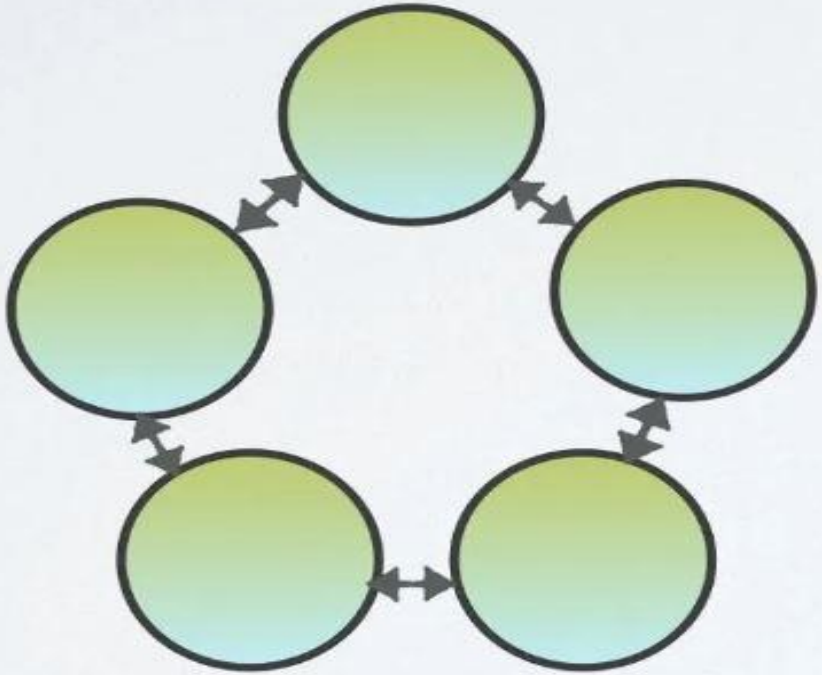
- The SOLID principles were introduced by Robert C. Martin in his 2000 paper "Design Principles and Design Patterns." These concepts were later built upon by Michael Feathers, who introduced us to the SOLID acronym.
- And in the last 20 years, these **five principles** have revolutionized the world of **object-oriented programming,** changing the way that we write software.

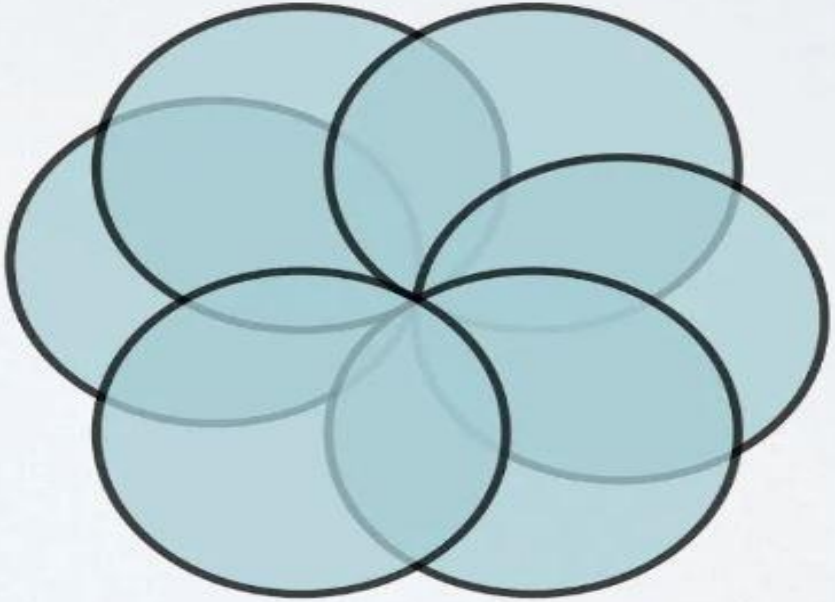
04-Why Use SOLID

- The Reason for SOLID Principles
  design principles encourage us to
  - Create more maintainable
  - Understandable
  - Flexible software.
  - As our applications grow in size, we can reduce their complexity



05-Class Coupling



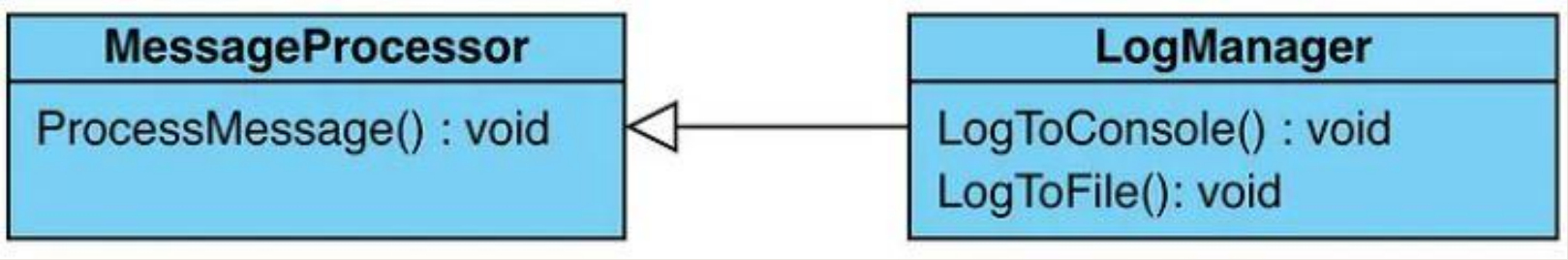


Loose Coupling Vs Tight Coupling

Tight Coupling:

Refers to a situation in software design where **two components** are tightly coupled. In this case, changes in the **internal structure of one class** can **affect the other class**, and the coupling between the two classes is tight.

Tight Coupling:



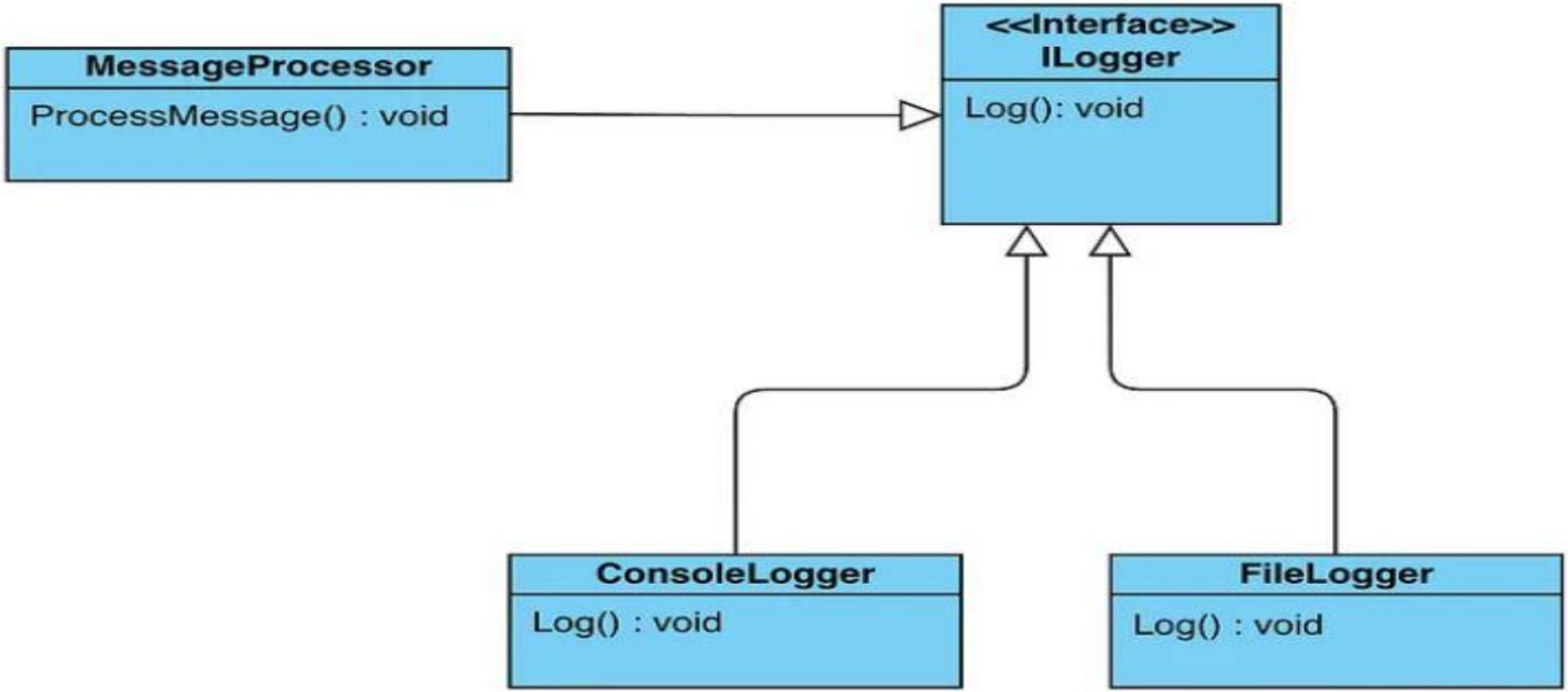
Loose Coupling:

Loose Coupling means **reducing dependencies** of a class that use a different class directly.

Benefits :

- **Flexibility**
- Ease of **Maintenance**
- **Independent Development**
- **Reusability**
- Ease of **Testing**
- **Modularity** and **Understandability**
- **Parallel Development**

Loose Coupling:



06-OOP, SOLID

- **Object Oriented Programming** is based on the idea of "objects".
  - Encapsulation
  - Abstraction
  - Inheritance
  - Polymorphism

- **SOLID**
  - is set of **Principles** which helps to **design** a "Solution" for a object oriented programming concepts
  - Basically design principles used to create **great software Architecture** to develop a better software

07-SOLID Principles

S	Single responsibility principle
O	Open/closed principle
L	Liskov substitution principle
I	Interface segregation principle
D	Dependency inversion principle

1.Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) states that a class should have only one reason to change, or in other words, it should have only one responsibility. This means that a class should have only one job to do, and it should do it well.

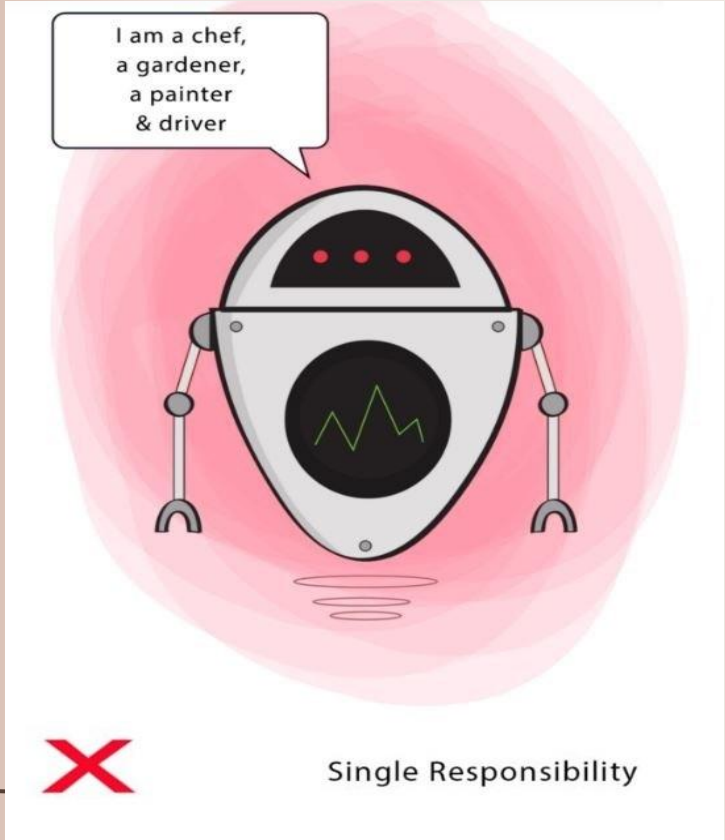
If a class has **too many responsibilities**,
it can become hard to understand,
maintain, and modify.

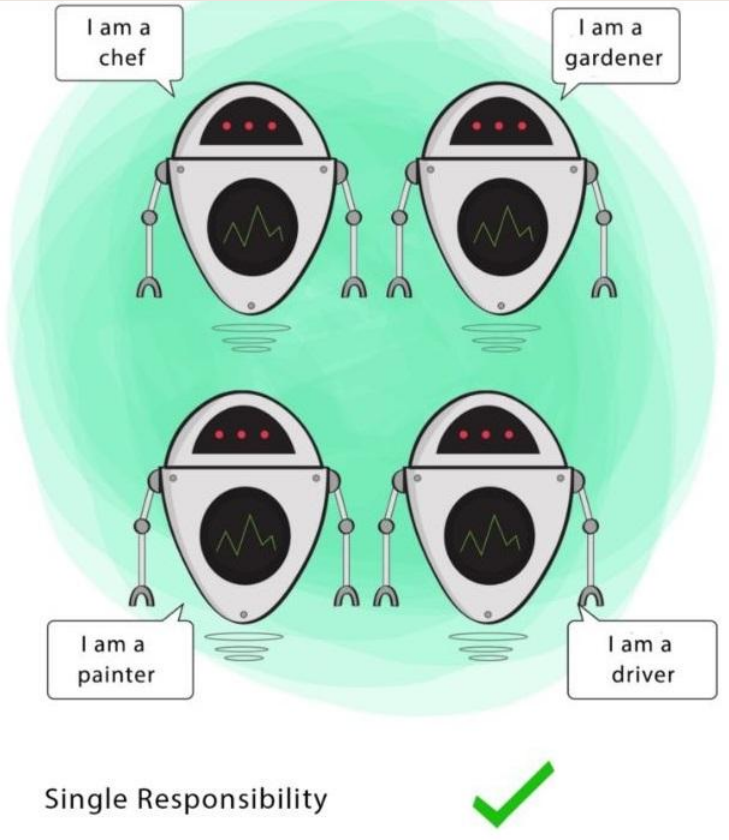
Changes to one responsibility can inadvertently affect another responsibility, leading to unintended consequences and bugs. By following SRP, we can create code that is **more modular**, **easier to understand**, and **less prone to errors**,

Divide and Conquer

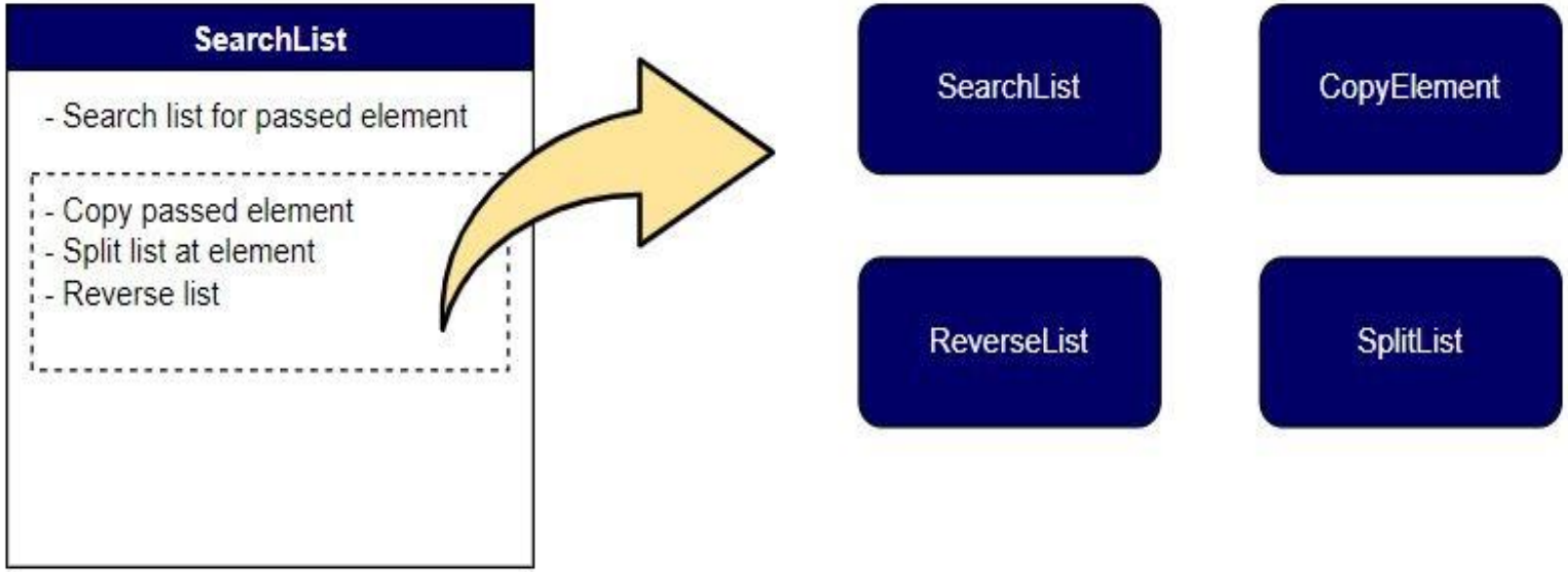
1.Single Responsibility Principle (SRP)

1.Single Responsibility Principle (SRP)





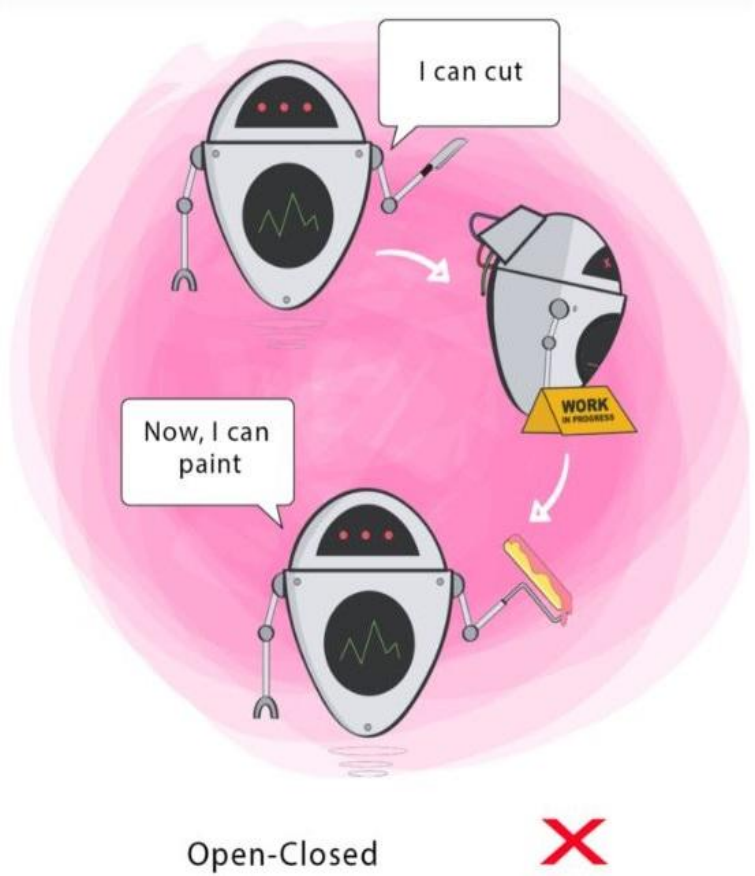
1.Single Responsibility Principle (SRP)



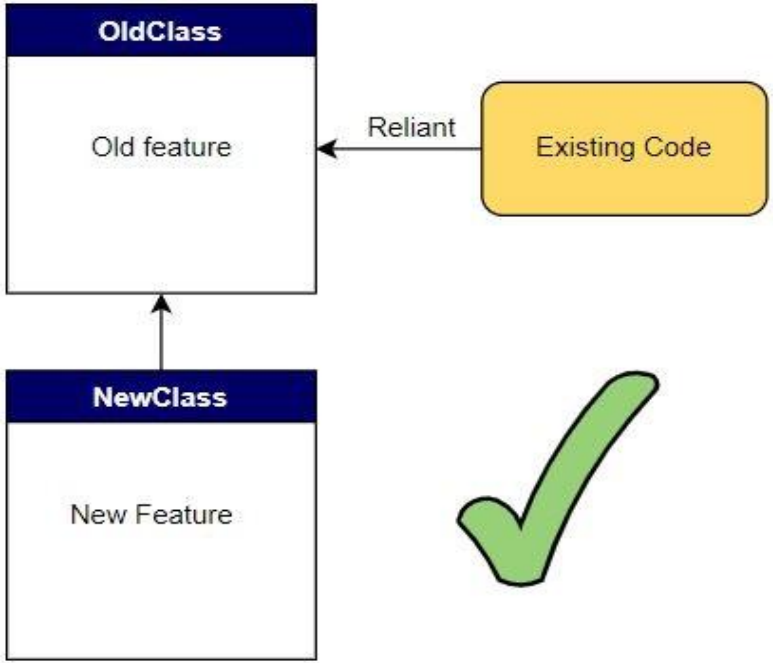
2.Open/Closed Principle (OCP)

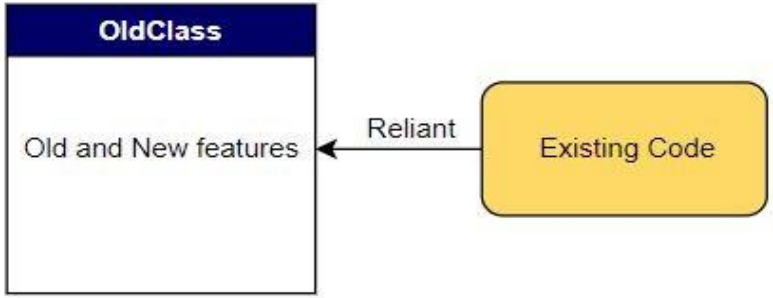
The Open-Closed Principle (OCP) states that **software entities (classes, modules, functions, and so on) should** **be open for extension but closed for modification.** This means that the behavior of a software entity can be extended **without modifying its source code**.
The OCP is essential because it promotes software extensibility and maintainability. By allowing software entities to be extended without modification, developers can **add new functionality without the risk of breaking existing code**. This results in code that is easier to maintain, extend, and reuse.

2.Open/Closed Principle (OCP)



2.Open/Closed Principle (OCP)



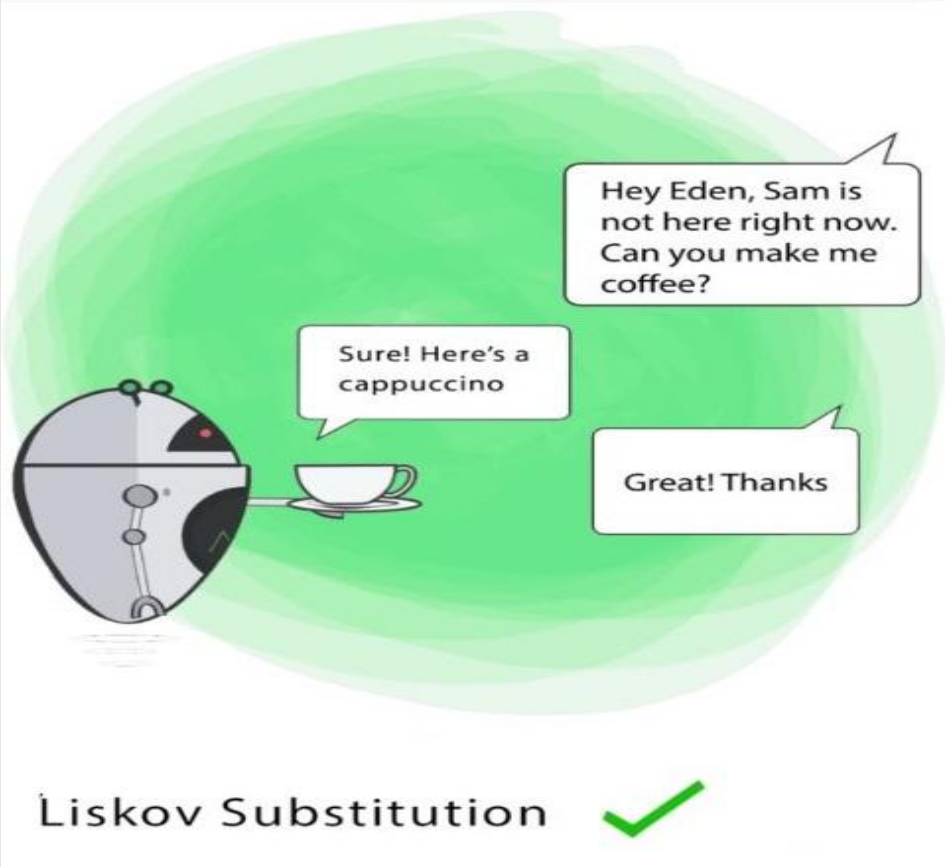


3.Liskov Substitution Principle (LSP)

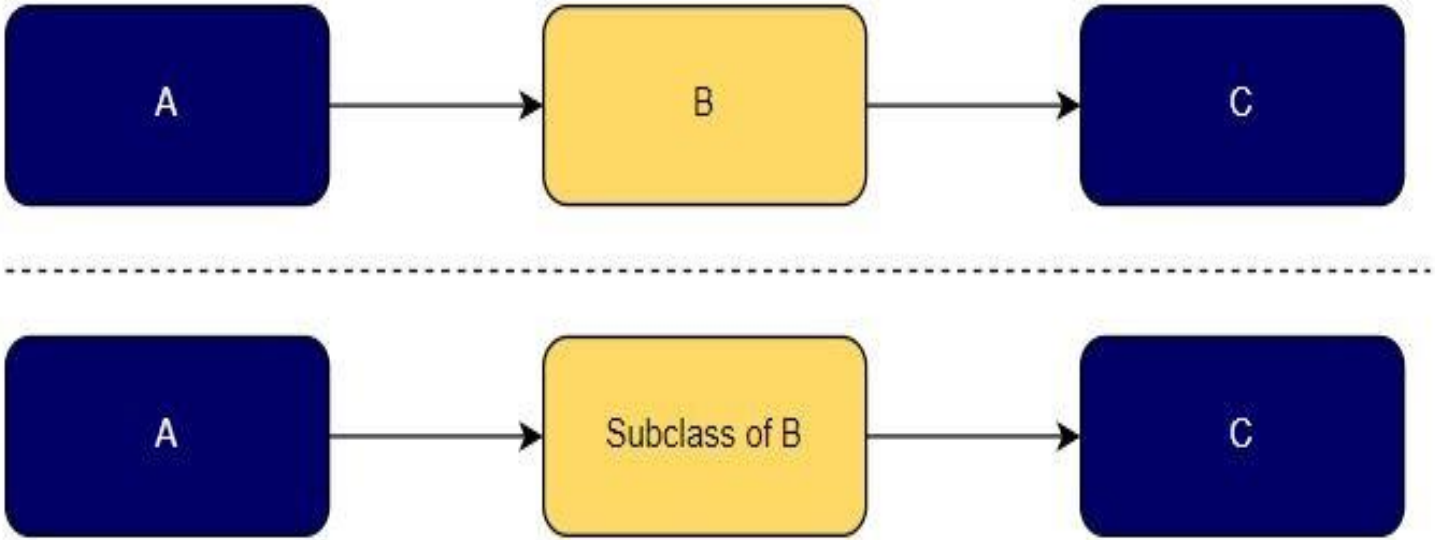
The Liskov Substitution Principle (LSP) states that any instance of a derived class should be substitutable for an instance of its base class without affecting the correctness of the program.
In other words, a derived class should behave like its base class in all contexts. In more simple terms, **if class A is a subtype (Child) of class B (Parent)**, you should be able to replace B **(Parent)**, with A **(Child)** without breaking the behavior of your program.
The importance of the LSP lies in its ability to ensure that the behavior of a program remains consistent and predictable when substituting objects of different classes. Violating the LSP can lead to unexpected behavior, bugs, and maintainability issues.

3.Liskov Substitution Principle (LSP)

3.Liskov Substitution Principle (LSP)



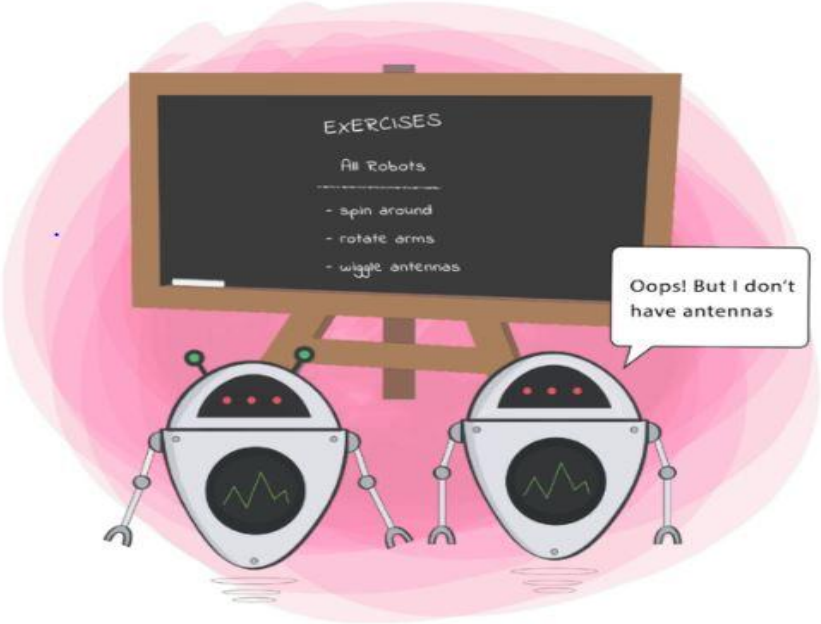
3.Liskov Substitution Principle (LSP)

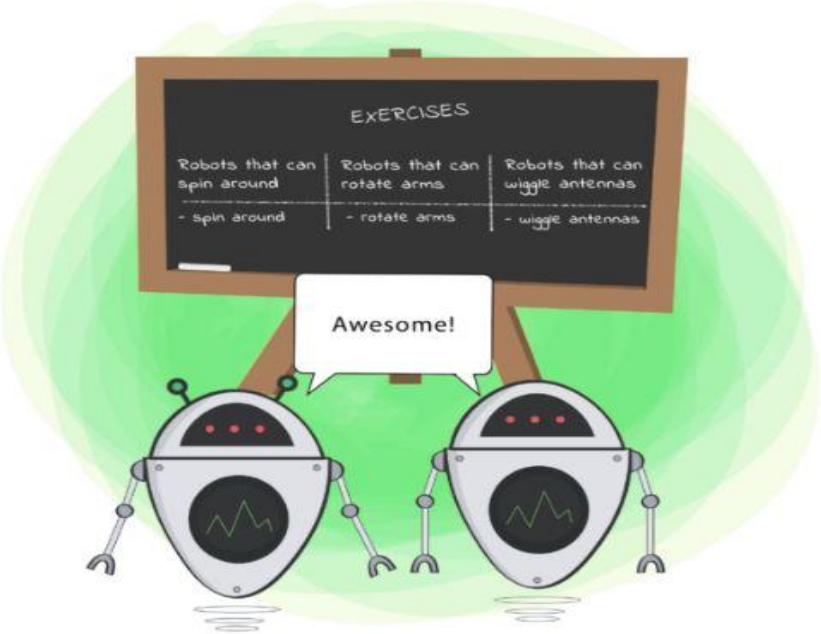


4. Interface Segregation Principle (ISP)

The Interface Segregation Principle (ISP) focuses on designing interfaces that are specific to their client's needs. It states that no client should be forced to depend on methods it does not use.
The principle suggests that **instead of creating a large interface that covers all the possible methods, it's better to create smaller, more focused interfaces for specific use cases**. This approach results in interfaces that are more cohesive and less coupled.

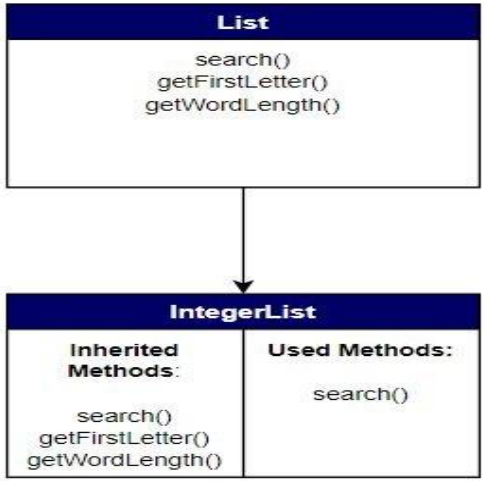
4. Interface Segregation Principle (ISP)





Interface Segregation

4. Interface Segregation Principle (ISP)



5. Dependency Inversion Principle (DIP)

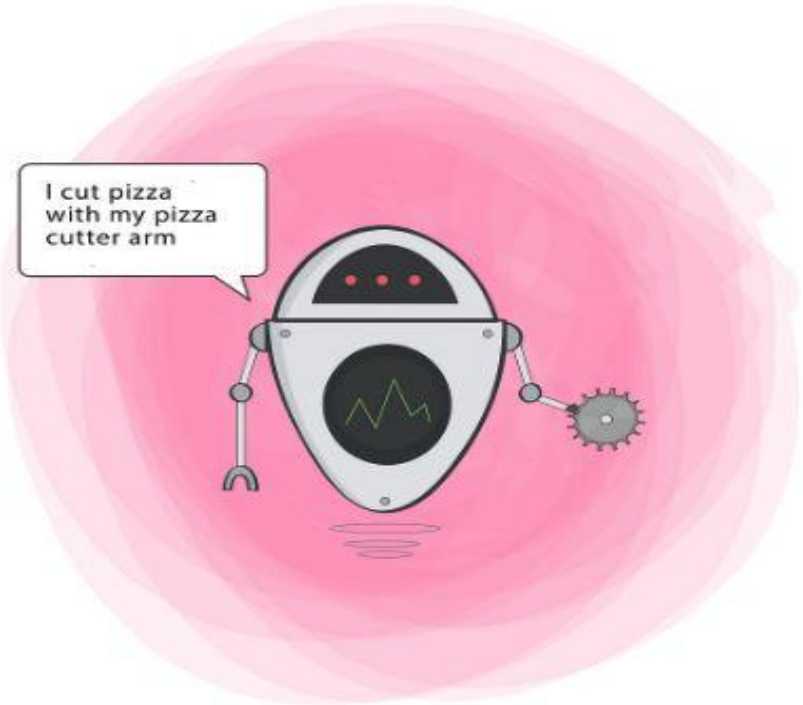
The Dependency Inversion Principle (DIP) states that high-level modules *should not* depend on low-level modules, but both should depend on abstractions. Abstractions should not depend on details – details should depend on abstractions.

This principle aims to reduce coupling between modules, increase modularity, and make the code easier to maintain, test, and extend.

high-level modules : Generally the **core business logic** of an application

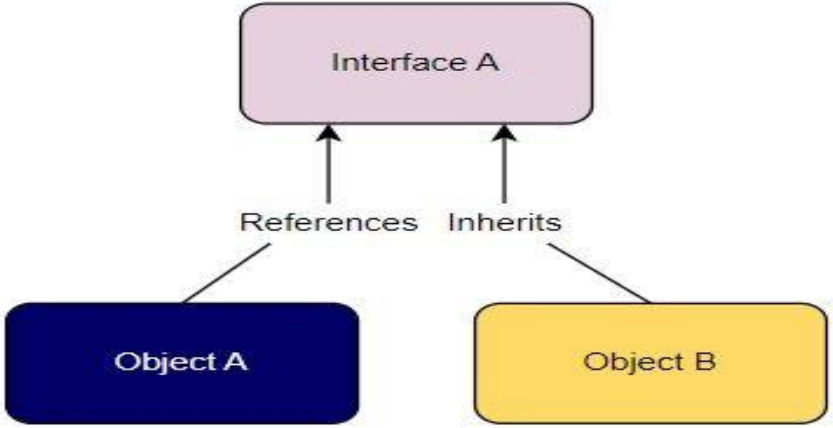
on low-level modules: Input/Output, database, file system, web API, or other external modules that interact with users

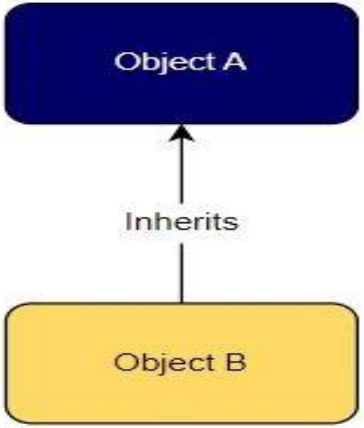
5. Dependency Inversion Principle (DIP).



Dependency Inversion

5. Dependency Inversion Principle (DIP).





Different Between
DI (Dependency Injection)
DIP (Dependency Inversion Principle)

- DI **(Dependency Injection)** is a technique to implement DIP, allowing dependencies to be injected into classes.
- DIP (**Dependency Inversion Principle)** outlines the principle of depending on abstractions rather *than concrete implementations*.

Usage Abstract vs Interface

Point	Abstract Class	Interface
Common/ Shared Logic	Yes	No
Multiple-Inheritance	No	Yes
Constructor	No Object So ? Even though you **cannot create an instance** of an abstract class directly, **the constructor still runs** when a derived (child) class is instantiated.	No
Fields	Yes	No
In-General if All equals	No	Yes

In-General (equals) Why Use Interface:

Reason	Explanation
Flexibility	Interface allows multiple inheritance. You can implement many interfaces together.
Loose Coupling	Code is less tied to a specific inheritance chain.
Better for Dependency Injection	Easier to mock and swap implementations in testing .
Future-proof	If later your system grows, you can still easily move from interface → abstract class, but hard the other way.
Clean Architecture Principle	Programming against abstractions (interface) is the default design best practice.

OOP-SOLID-Design Pattern

Concept	Simple Meaning	Main Goal	Examples
OOP	A way to ***write code using objects and classes***	Organize code around ***real things and their behavior***	Encapsulation, Inheritance, Polymorphism, Abstraction
SOLID	5 design ***rules*** for writing good OOP code	Make ***code easier to maintain, test, and extend***	Single Responsibility, Open/Closed, Liskov, Interface Segregation, Dependency Inversion
Design Patterns	***Common*** ready-made ***solutions for repeated coding problems***	Solve specific ***design problems in a standard way***	Singleton, Factory, Repository, Strategy, Observer

OOP-SOLID-Design Pattern

Simple relationship:
OOP = the *foundation*
SOLID = *rules* for *using OOP correctly*
Design Patterns = *practical solutions* built using OOP and following SOLID

Example:
If you build a **payment system**:
OOP : create *classes* like Payment, CreditCardPayment, PayPalPayment
SOLID : make each class have *one responsibility and depend on interfaces*
Design Pattern : use the *Strategy* Pattern to *switch between payment methods easily*

Questions ?

Thank you